

GUIDE DE PRÉPARATION · NIVEAU 1

OpenShift Container Platform

Tout ce qu'il faut savoir pour le cours DO180 et l'examen EX180
— Red Hat Certified Technologist in OpenShift — avec 30
exercices pratiques sur ton propre cluster.

Lab Single Node OpenShift 4.22 · `ocp.lab.example.com`

Version 30 septembre 2026

DO180

EX180

oc

OCP 4.22

Document d'étude personnel, non officiel et non affilié à Red Hat, Inc. « Red Hat » et « OpenShift » sont des marques de Red Hat, Inc. Les objectifs d'examen cités proviennent des pages publiques de Red Hat (consultées le 30/09/2026) et peuvent évoluer : vérifie toujours la version officielle avant de réserver.

SOMMAIRE

Contenu du guide

01	Le parcours de certification OpenShift	3
02	L'examen EX180 en détail	5
03	Le cours DO180 chapitre par chapitre	7
04	Les concepts à maîtriser	9
05	Ton lab : fiche technique et accès	12
06	Aide-mémoire de la commande <code>oc</code>	14
07	Exercices pratiques (30)	16
08	Examen blanc (2 heures)	33
09	Stratégie d'examen et pièges classiques	35
10	Plan de révision, checklist et ressources	37

COMMENT UTILISER CE GUIDE

Lis les chapitres 1 à 4 une fois pour avoir la vue d'ensemble. Ensuite, travaille **uniquement au clavier** : fais chaque exercice du chapitre 7 dans la console web, puis refais-le avec `oc` sans regarder la solution. Quand tu réussis l'examen blanc (chapitre 8) en moins de 2 heures, tu es prêt.

CONVENTIONS

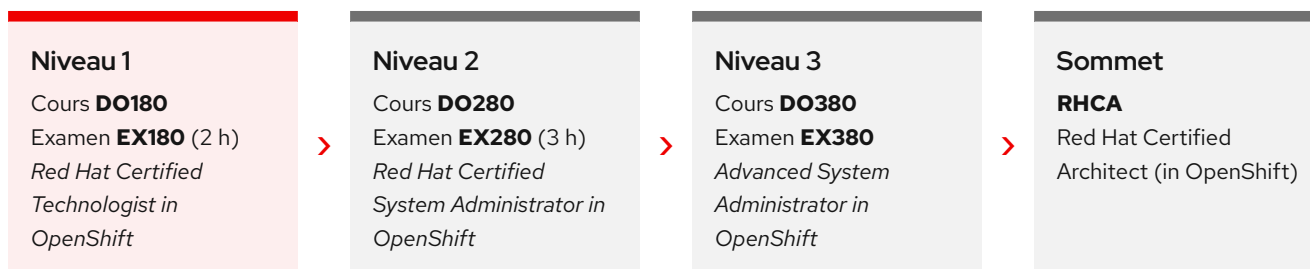
- `$` : commande à lancer sur ton poste (Fedora) avec `oc` connecté au cluster.
- `<nom>` : valeur à remplacer.
- Chaque exercice est rattaché à un **objectif officiel EX180** (étiquette en haut à droite) et à un chapitre du DO180.

CHAPITRE 01

Le parcours de certification OpenShift

Où se situe le « niveau 1 » et ce qui vient après.

Red Hat organise ses certifications OpenShift en paliers. Chaque palier associe un **cours** (code DO...) et un **examen pratique** (code EX...). Le cours n'est pas obligatoire : seul l'examen donne la certification.



■ Ce que valide chaque niveau

Niveau	Cours / examen	Ce qu'on attend de toi
1 – Technologist	DO180 → EX180	Déployer, exposer, configurer, stocker, fiabiliser et dépanner des applications sur un cluster existant. Point de vue « opérateur d'applications ».
2 – System Administrator	DO280 → EX280	Administrer le cluster : authentification (HTPasswd), utilisateurs, groupes, RBAC, NetworkPolicies, TLS, quotas, LimitRanges, templates de projet, Operators, SCC, ServiceAccounts, Jobs/ CronJobs, Kustomize, Helm. Inclut toutes les tâches du niveau 1.
3 – Advanced	DO380 → EX380	Exploitation à grande échelle : authentification entreprise, sauvegarde, GitOps, logging, mise à l'échelle.

CHANGEMENT DE NOMS EN 2026

Red Hat a renommé ses titres. L'EX280 s'appelle désormais **Red Hat Certified System Administrator in OpenShift** (anciennement « Certified OpenShift Administrator » / « Specialist in OpenShift Administration »). Le niveau 1 correspond au titre **Red Hat Certified Technologist in OpenShift** (EX180). Tu verras encore les anciens noms chez les revendeurs et dans les annonces d'emploi : ce sont les mêmes examens.

■ Pourquoi le niveau 1 est une bonne première marche

- **Il est la base obligatoire de l'EX280** : les objectifs officiels de l'EX280 demandent explicitement de savoir faire toutes les tâches du Technologist.
- **Il couvre exactement ce qu'un stagiaire DevOps/cloud fait au quotidien** : déployer une appli, l'exposer, lui brancher de la config et du stockage, la rendre robuste, la mettre à jour.

- **C'est un examen pratique** : pas de QCM, tu prouves que tu sais faire. Les employeurs le savent.
- Si tu as déjà pratiqué Docker ou K3s, 80 % des notions Kubernetes sont transférables.

■ Coûts et formats (indicatifs)

Élément	Information
Examen EX180 seul	Environ 530 € HT chez les revendeurs européens (tarif indicatif, variable selon le pays).
Cours DO180	3 à 5 jours selon le format ($\approx$ 32 h en classe virtuelle). Basé sur OpenShift 4.18 dans sa version actuelle.
Red Hat Learning Subscription	Accès d'un an à tous les cours en ligne, labs, et au moins un passage d'examen selon la formule.
Évaluation gratuite	<code>skills.ole.redhat.com</code> – test de niveau gratuit pour savoir par quel cours commencer.
Formation proposée par l'entreprise	Demande si le passage de l'examen est inclus (souvent vendu à part), la date limite pour le passer, et s'il y a une suite prévue vers le DO280.

CHAPITRE 02

L'examen EX180 en détail

Format, règles et liste complète des objectifs officiels.

■ Fiche d'identité

Critère	Détail
Titre obtenu	Red Hat Certified Technologist in OpenShift
Code	EX180
Durée	2 heures , en une seule partie
Type	100 % pratique : tu réalises des tâches sur un cluster réel ; aucune question à choix multiples
Version du produit	OpenShift Container Platform 4.22 selon la page officielle actuelle (des revendeurs affichent encore 4.18). Plusieurs versions de l'examen coexistent : la tienne est indiquée dans le document « Red Hat Certification Exam Objectives by Version » du LMS Red Hat.
Documentation	La documentation produit est fournie pendant l'épreuve. Aucune autre aide (Internet, notes, IA) n'est autorisée.
Résultats	Envoyés par e-mail par Red Hat Certification Central, en général sous 3 jours ouvrés (US) . Seul le score total est communiqué, sans détail par tâche.
Règle de persistance	Tout ce que tu configures doit continuer à fonctionner pendant le cycle de vie normal de la plateforme, sans intervention manuelle (redémarrage de pod, de nœud...).
Prérequis recommandés	RH124 + RH134 (bases Linux, niveau RHCSA) ou expérience équivalente ; DO180 ou expérience équivalente. Rien n'est obligatoire.
Public visé	Admins système passant aux conteneurs, développeurs qui déploient sur OpenShift, ingénieurs DevOps, SRE, platform engineers.

À LIRE AVANT DE RÉSERVER

Consulte le **Red Hat Certification Program Guide** (docs.redhat.com) : pièces d'identité, conditions de l'examen à distance (webcam, pièce vide, clé USB de démarrage pour le « remote exam »), politique d'annulation, délai de repassage.

■ Objectifs officiels complets

Liste intégrale publiée par Red Hat, traduite, avec le libellé original et les exercices de ce guide qui s'y rattachent.

Objectif (FR)	Libellé officiel (EN)	Exercices
1. Gérer OpenShift Container Platform		
Installer et configurer l'outil CLI OpenShift	Install and configure the OpenShift CLI tool	E01, E02
Inspecter les ressources Kubernetes via la CLI et l'interface web	Inspect Kubernetes resources using both the CLI and the Web UI	E03 à E07
Collecter les données de support d'un cluster pour le support Red Hat	Collect cluster support data from an OpenShift cluster for Red Hat support	E08
2. Déployer des applications sur OpenShift		
Créer des conteneurs Linux et des pods Kubernetes	Create Linux containers and Kubernetes pods	E09 à E12
Trouver et inspecter des images de conteneurs	Find and inspect container images	E13, E14
Déployer des applications depuis une image et depuis un template	Deploy applications from an image and from a template	E15, E16, E17
Mettre à l'échelle et exposer les applications à l'extérieur	Scale and expose applications to external access	E18 à E21
3. Gérer le stockage de la configuration et des données		
Externaliser la configuration des applications	Externalize the configuration of applications	E22, E23
Provisionner des volumes de données persistants	Provision persistent data volumes	E24, E25
4. Configurer les applications pour la fiabilité		
Configurer les sondes de santé (health probes)	Configure application health probes	E26
Réserver et limiter les ressources de calcul	Reserve and limit compute capacity for applications	E27
Configurer l'autoscaling des applications	Configure application autoscaling	E28
Hors liste EX180 mais au programme du DO180		
Gérer les mises à jour et retours arrière d'une application	DO180 ch. 7 – Manage Application Updates	E29, E30

NOTE

Red Hat se réserve le droit de modifier les objectifs (changements annoncés à l'avance). Relis la page officielle de l'EX180 la semaine de ton examen.

CHAPITRE 03

Le cours DO180 chapitre par chapitre

Red Hat OpenShift Administration I – Operating a Production Cluster (aussi appelé « Containers & Kubernetes »).

Le DO180 forme les administrateurs à gérer des charges Kubernetes et à collaborer avec développeurs, DevOps et SRE pour garantir la disponibilité des applications. Les compétences s'appliquent à toutes les variantes d'OpenShift : OpenShift Container Platform (ton lab), ROSA (AWS) et ARO (Azure).

■ Objectifs du cours

- Gérer un cluster OpenShift depuis la CLI et depuis la console web.
- Déployer des applications à partir d'images de conteneurs, de templates et de manifestes Kubernetes.
- Dépanner la connectivité réseau entre applications, à l'intérieur et à l'extérieur du cluster.
- Connecter les charges Kubernetes à du stockage pour les données applicatives.
- Configurer les charges pour la haute disponibilité et la fiabilité.
- Gérer les mises à jour des images, des paramètres et des manifestes d'une application.

■ Plan détaillé

#	Chapitre	Ce que tu y apprends	Notions clés
1	Introduction à Kubernetes et OpenShift	Identifier les services du cluster Kubernetes et de la plateforme OpenShift, et les surveiller depuis la console web.	Control plane, nœuds, Operators, ClusterOperators, console, alertes
2	CLI et API Kubernetes / OpenShift	Accéder au cluster en ligne de commande et interroger les ressources de l'API pour évaluer sa santé.	<code>oc login</code> , <code>get</code> , <code>describe</code> , <code>explain</code> , <code>jsonpath</code> , <code>must-gather</code>
3	Exécuter des applications en conteneurs et en pods	Lancer et dépanner des applications conteneurisées sous forme de pods non gérés.	Pods, images, registres, <code>oc run</code> , <code>exec</code> , <code>logs</code> , <code>debug</code> , SCC
4	Déployer des applications gérées et en réseau	Déployer des applications et les exposer à l'intérieur et à l'extérieur du cluster.	Deployments, ReplicaSets, Services, Routes, Ingress, templates, labels
5	Gérer le stockage de la configuration et des données	Externaliser la configuration dans des ressources Kubernetes et provisionner des volumes persistants.	ConfigMaps, Secrets, PV, PVC, StorageClass, modes d'accès
6	Configurer les applications pour la fiabilité	Rendre les applications hautement disponibles et résilientes.	Probes, requests/limits, HPA, réplicas

#	Chapitre	Ce que tu y apprends	Notions clés
7	Gérer les mises à jour des applications	Gérer des mises à jour reproductibles et des retours arrière du code et de la configuration.	Tags et digests, ImageStreams, rollouts, stratégies, triggers

CORRESPONDANCE AVEC TON EXPÉRIENCE

Chapitres 3 à 6 : du Kubernetes pur, que tu pratiques déjà avec K3s (`kubect1` fonctionne aussi sur OpenShift). Ce qui est **spécifique à OpenShift** et qu'il faut travailler en priorité : Projects, Routes, SCC, templates, ImageStreams, `oc new-app`, `oc set ...`, la console web et `oc adm must-gather`.

CHAPITRE 04

Les concepts à maîtriser

Le vocabulaire et l'architecture que l'examen suppose acquis.

■ Architecture d'un cluster OpenShift

Composant	Rôle
Control plane	Le « cerveau » : API server (point d'entrée unique, port 6443), etcd (base clé/valeur de l'état du cluster), scheduler (place les pods), controller manager (réconcilie l'état réel avec l'état voulu).
Workers (compute)	Exécutent les pods applicatifs. Dans ton SNO, un seul nœud cumule les rôles control plane et worker.
RHCOS	Red Hat Enterprise Linux CoreOS : OS immuable des nœuds, géré par le cluster lui-même (Machine Config Operator). On ne s'y connecte qu'en dépannage (<code>oc debug node/...</code>).
CRI-O	Moteur d'exécution des conteneurs sur les nœuds (pas Docker).
OVN-Kubernetes	Réseau logiciel (SDN) du cluster : réseau des pods (10.128.0.0/14 par défaut) et des services (172.30.0.0/16).
Ingress Controller (router)	HAProxy qui reçoit le trafic HTTP/HTTPS externe (ports 80/443) et le distribue aux Routes. D'où le DNS wildcard <code>*.apps</code> .
Operators / ClusterOperators	Programmes qui installent et maintiennent chaque composant. <code>oc get clusteroperators</code> donne la santé de la plateforme. OLM (Operator Lifecycle Manager) gère les Operators additionnels via OperatorHub.
Registre interne	Registre d'images intégré (<code>image-registry.openshift-image-registry.svc:5000</code>), utilisé par les ImageStreams et les builds.
Monitoring	Prometheus + Alertmanager préinstallés ; alertes et métriques visibles dans la console (Observe).

■ Ce qu'OpenShift ajoute à Kubernetes

Kubernetes	OpenShift	Différence à retenir
Namespace	Project	Un namespace avec des annotations et un modèle de création ; <code>oc new-project</code> te bascule dedans.
Ingress	Route	Exposition HTTP(S) native ; types edge, passthrough, re-encrypt. <code>oc expose svc/x</code> .

Kubernetes	OpenShift	Différence à retenir
Pod Security	SCC (Security Context Constraints)	Par défaut (<code>restricted-v2</code>), un conteneur ne tourne pas en root et reçoit un UID aléatoire. Beaucoup d'images Docker Hub échouent pour cette raison.
–	Templates	Ensemble paramétrable de ressources (ex. <code>mysql-persistent</code>), instancié avec <code>oc new-app --template</code> ou <code>oc process</code> .
–	ImageStream	Pointeur vers des images (tags), capable de déclencher un redéploiement quand l'image change.
–	BuildConfig / S2I	Construction d'images dans le cluster (Source-to-Image, Dockerfile). Hors objectifs EX180, mais utile.
kubeconfig manuel	OAuth intégré	<code>oc login</code> avec utilisateur/mot de passe ou token ; fournisseurs d'identité (HTPasswd, LDAP...).
<code>kubectl</code>	<code>oc</code>	Surensemble de kubectl : toutes les commandes kubectl + <code>new-app</code> , <code>new-project</code> , <code>expose</code> , <code>set</code> , <code>rsh</code> , <code>adm</code> , <code>process</code> ...

■ Objets Kubernetes essentiels

Objet	À quoi il sert	Abréviation
Pod	Plus petite unité : un ou plusieurs conteneurs partageant réseau et volumes.	<code>po</code>
Deployment	Déclare l'état voulu (image, réplicas) ; crée des ReplicaSets ; gère les rollouts.	<code>deploy</code>
ReplicaSet	Maintient N pods identiques (sélection par labels).	<code>rs</code>
Service	IP et nom DNS stables devant un groupe de pods (ClusterIP, NodePort, LoadBalancer).	<code>svc</code>
Route	Exposition externe HTTP(S) d'un Service.	<code>route</code>
ConfigMap / Secret	Configuration non sensible / sensible, injectée en variables ou en fichiers.	<code>cm</code> / <code>secret</code>
PV / PVC	Volume persistant / demande de volume par une appli.	<code>pv</code> / <code>pvc</code>
StorageClass	Type de stockage provisionné dynamiquement (chez toi : LVMS).	<code>sc</code>
HorizontalPodAutoscaler	Ajuste le nombre de réplicas selon la charge CPU/mémoire.	<code>hpa</code>
Job / CronJob	Tâche ponctuelle / planifiée (niveau 2).	<code>job</code> / <code>cj</code>
StatefulSet / DaemonSet	Pods avec identité stable / un pod par nœud.	<code>sts</code> / <code>ds</code>

■ Labels, sélecteurs et flux d'une requête

Tout est relié par des **labels** : le Deployment crée des pods portant `app=web` ; le Service sélectionne `app=web` pour trouver ses endpoints ; la Route pointe vers le Service. Si un maillon ne correspond pas, l'application est injoignable.

```
Navigateur → DNS *.apps → Router (HAProxy :443) → Route → Service (ClusterIP)
→ Endpoints → Pods (app=web)
```

■ Probes, ressources et autoscaling

Mécanisme	Effet
Readiness probe	Tant qu'elle échoue, le pod est retiré des endpoints du Service (pas de trafic). Pas de redémarrage.
Liveness probe	Si elle échoue, le conteneur est redémarré .
Startup probe	Protège les applis lentes au démarrage : liveness/readiness ne démarrent qu'après son succès.
Requests	Ressources réservées : utilisées par le scheduler et par l'HPA (le % CPU est calculé sur la request).
Limits	Plafond : dépassement CPU = bridé ; dépassement mémoire = conteneur tué (OOMKilled).
HPA	Nécessite des requests définies ; ajoute/retire des répliques entre min et max.

■ États de pods à savoir diagnostiquer

État	Cause probable	Premier réflexe
<code>Pending</code>	Pas de nœud avec assez de ressources, PVC non lié, quota atteint.	<code>oc describe pod</code> (Events)
<code>ImagePullBackOff</code> / <code>ErrImagePull</code>	Nom/tag d'image faux, registre privé sans secret, registre injoignable.	<code>oc describe pod</code> , vérifier l'image
<code>CrashLoopBackOff</code>	Le processus quitte : erreur de config, variable manquante, droits root refusés par la SCC , port < 1024.	<code>oc logs</code> , <code>oc logs --previous</code>
<code>OOMKilled</code>	Limite mémoire trop basse.	Augmenter <code>limits.memory</code>
<code>Running</code> mais O/1 Ready	Readiness probe en échec (mauvais port/chemin).	<code>oc describe pod</code>
<code>CreateContainerConfigError</code>	ConfigMap ou Secret référencé inexistant.	<code>oc get cm,secret</code>

CHAPITRE 05

Ton lab : fiche technique et accès

Le cluster installé le 30 septembre 2026 avec l'Assisted Installer.

■ Fiche technique

Élément	Valeur
Type	Single Node OpenShift (SNO) – control plane + worker sur un seul nœud
Version	OpenShift Container Platform 4.22.15, réseau OVN-Kubernetes
Licence	Essai de 60 jours (à surveiller ; OKD SNO en alternative gratuite ensuite)
VM Proxmox	VM 200 <code>ocp-sno</code> sur pve-node1 · 8 vCPU (type host) · 16 Go RAM (ballooning off) · disque 120 Go (système) + 50 Go (LVMS)
Réseau	VLAN serveurs · IP fixe 10.0.50.10 (static mapping pfSense, MAC <code>52:54:00:12:34:56</code>)
Nom du cluster / domaine	<code>ocp</code> / <code>lab.example.com</code>
API	<code>https://api.ocp.lab.example.com:6443</code>
Console	<code>https://console-openshift-console.apps.ocp.lab.example.com</code>
Applications	<code>*.apps.ocp.lab.example.com</code> (wildcard DNS dans pfSense)
Stockage	Operator LVM Storage (LVMS) ; StorageClass attendue : <code>lvms-vg1</code> (à vérifier avec <code>oc get sc</code>)
Accès SSH de secours	<code>ssh core@10.0.50.10</code> avec la clé ed25519 du PC Admin
Réseaux internes	Pods 10.128.0.0/14 · Services 172.30.0.0/16 · machine 10.0.50.0/24

■ DNS dans pfSense (rappel)

```
# Services > DNS Resolver > Host Overrides
api.ocp.lab.example.com      A  10.0.50.10
api-int.ocp.lab.example.com  A  10.0.50.10

# Services > DNS Resolver > General Settings > Custom options
server:
local-zone: "apps.ocp.lab.example.com" redirect
local-data: "apps.ocp.lab.example.com A 10.0.50.10"
```

■ Accès à distance par le VPN

Le domaine `lab.example.com` n'existe que dans pfSense : le poste doit l'interroger **à travers le tunnel**.

Fedora (NetworkManager)

```
$ sudo nmcli con mod "VPN-Lab" ipv4.dns 10.0.10.1 ipv4.dns-search "~lab.example.com"
$ nmcli con down "VPN-Lab" && nmcli con up "VPN-Lab"
$ resolvectl query console-openshift-console.apps.ocp.lab.example.com # → 10.0.50.10
```

Windows (OpenVPN Connect)

Si l'application envoie ce domaine vers la box locale (règle NRPT, fréquent quand la box utilise le même suffixe), ajoute une règle plus précise (PowerShell administrateur) :

```
Add-DnsClientNrptRule -Namespace ".lab.example.com" -NameServers 10.0.10.1
ipconfig /flushdns
```

LEÇON RETENUE

Évite `.lan` pour un lab : beaucoup de box l'utilisent déjà, ce qui crée des conflits DNS via le VPN. Préfère `home.arpa` ou un sous-domaine que tu possèdes. Le domaine d'un cluster OpenShift ne peut pas être changé après installation.

■ Connexion en CLI

```
# Installer oc (une fois)
$ curl -LO https://mirror.openshift.com/pub/openshift-v4/clients/ocp/stable/openshift-client-linux.tar.gz
$ tar xzf openshift-client-linux.tar.gz oc kubectl && sudo mv oc kubectl /usr/local/bin/

# Se connecter (console > ton nom > Copy login command > Display Token)
$ oc login --token=sha256~xxxx --server=https://api.ocp.lab.example.com:6443

# Ou avec un utilisateur (après l'exercice E02)
$ oc login -u student https://api.ocp.lab.example.com:6443
```

SPÉCIFICITÉS D'UN SNO À CONNAÎTRE

- **Un seul nœud** : les exercices d'anti-affinité ou de répartition sur plusieurs nœuds ne sont pas reproductibles ; le reste l'est.
- **Ressources limitées** : garde des requests modestes (100m CPU, 128Mi) sinon les pods restent `Pending`.
- LVMS fournit du **RWO** (ReadWriteOnce) uniquement : pas de RWX.
- Après un arrêt de la VM, compte 5 à 10 minutes avant que tous les ClusterOperators soient disponibles.

CHAPITRE 06

Aide-mémoire de la commande oc

Les commandes à connaître par cœur pour l'examen.

Connexion et contexte

```
oc login -u <user> <api-url>      oc whoami      oc whoami --show-console
oc project                        oc projects    oc project <nom>
oc new-project <nom>              oc delete project <nom>
oc status                          oc version     oc api-resources
```

Lire et filtrer

```
oc get pods -o wide                oc get all      oc get pods --show-labels
oc get pods -l app=web             oc get pods -A  oc get events --sort-
by=.lastTimestamp
oc describe pod/<nom>              oc get deploy/web -o yaml
oc get pods -o jsonpath='{.items[*].metadata.name}'
oc get pods -o custom-columns=NOM:.metadata.name,IP:.status.podIP,NODE:.spec.nodeName
oc get deploy web -o jsonpath='{.spec.template.spec.containers[0].image}{ "\n"}'
oc explain deployment.spec.template.spec.containers.readinessProbe # doc intégrée
```

Santé du cluster

```
oc get nodes                      oc get clusteroperators (co)    oc get clusterversion
oc adm top nodes                  oc adm top pods -n <ns>        oc describe node <nœud>
oc adm must-gather --dest-dir=./mg # données pour le support
oc adm inspect clusteroperator/ingress --dest-dir=./inspect # collecte ciblée
```

Pods et conteneurs

```
oc run <nom> --image=<image>      # pod non géré
oc run -it --rm tmp --image=registry.access.redhat.com/ubi9/ubi -- bash
oc logs <pod> [-c conteneur] [-f] [--previous]
oc exec <pod> -- <commande>      oc rsh <pod>
oc debug deploy/<nom>             oc debug node/<nœud> # puis chroot /host
oc cp <pod>:/chemin ./local      oc port-forward <pod> 8080:8080
```

Images

```
oc image info <image>            skopeo inspect docker://<image>
oc get is -n openshift           oc get istag -n openshift
oc import-image <is>:<tag> --from=<image> --confirm
oc tag <source> <is>:<tag>      podman run / images / inspect (sur ton poste)
```

Déployer

```
oc new-app --image=<image> [--name=x]      oc new-app --template=<tpl> -p PARAM=val
oc create deployment <nom> --image=<image> --replicas=2
oc get templates -n openshift              oc process --parameters <tpl> -n openshift
oc process openshift//<tpl> -p A=b | oc apply -f -
oc apply -f fichier.yaml                   oc create ... --dry-run=client -o yaml > f.yaml
```

Exposer et mettre à l'échelle

```
oc expose deploy/<nom> --port=8080          # crée le Service
oc expose svc/<nom> [--hostname=x.apps.ocp.lab.example.com] # crée la Route HTTP
oc create route edge <nom> --service=<svc>   # Route HTTPS (TLS au router)
oc scale deploy/<nom> --replicas=3           oc get route <nom> -o jsonpath='{.spec.host}'
```

Configuration et stockage

```
oc create configmap <cm> --from-literal=K=v --from-file=app.conf
oc create secret generic <s> --from-literal=password=xxx
oc set env deploy/<nom> --from=configmap/<cm>      oc set env deploy/<nom> --list
oc set env deploy/<nom> --from=secret/<s> --prefix=DB_
oc set volumes deploy/<nom> --add --type=configmap --configmap-name=<cm> --mount-path=/etc/app
oc set volumes deploy/<nom> --add --name=data --type=pvc --claim-name=<pvc> \
  --claim-size=1Gi --claim-mode=rwo --claim-class=lvms-vg1 --mount-path=/data
oc get pv,pvc,sc
```

Fiabilité

```
oc set probe deploy/<nom> --readiness --get-url=http://:8080/ --initial-delay-seconds=5
oc set probe deploy/<nom> --liveness --open-tcp=8080 --period-seconds=10
oc set resources deploy/<nom> --requests=cpu=100m,memory=128Mi --limits=cpu=500m,memory=256Mi
oc autoscale deploy/<nom> --min=1 --max=5 --cpu-percent=70      oc get hpa
```

Mises à jour

```
oc set image deploy/<nom> <conteneur>=<image>:<tag>
oc rollout status deploy/<nom>      oc rollout history deploy/<nom>
oc rollout undo deploy/<nom> [--to-revision=N]
oc rollout pause|resume|restart deploy/<nom>
oc set triggers deploy/<nom> --from-image=<is>:<tag> -c <conteneur>
```

ASTUCE EXAMEN

Active l'autocomplétion : `source <(oc completion bash)>` (ou `oc completion fish | source` sur ton poste). Et quand tu ne connais pas un champ YAML : `oc explain <objet>.<chemin> --recursive`.

CHAPITRE 07

Exercices pratiques (30)

Dans l'ordre du DO180. Pour chaque exercice : objectif, énoncé, vérification, puis solution.

RÈGLE DU JEU

Fais l'exercice **sans lire la solution**. Si tu bloques plus de 10 minutes, lis-la, puis supprime le projet et recommence de zéro le lendemain. Un exercice n'est acquis que lorsque tu le réussis sans aide.

■ Bloc A – Gérer OpenShift (DO180 ch. 1 et 2)

E01 Installer et configurer oc

EX180 · 1.1

Disposer d'un client oc fonctionnel, avec autocomplétion.

ÉNONCÉ

1. Installe `oc` sur ton poste depuis la console web (menu ? > Command Line Tools) ou depuis le miroir officiel.
2. Active l'autocomplétion dans ton shell.
3. Connecte-toi avec `kubeadmin` et affiche l'URL de la console.

VÉRIFICATION

`oc version` affiche client et serveur ; `oc whoami --show-console` renvoie l'URL de la console.

SOLUTION

```
$ oc version
$ oc completion fish | source          # bash : source <(oc completion bash)
$ oc login -u kubeadmin https://api.ocp.lab.example.com:6443
$ oc whoami; oc whoami --show-console
```

E02 Créer ton utilisateur (HTPasswd)

Prérequis lab · EX280

Ne plus travailler avec `kubeadmin`. (C'est un objectif du niveau 2, mais indispensable pour ton lab.)

ÉNONCÉ

1. Crée les utilisateurs `student` (admin) et `dev` (développeur) dans un fichier `htpasswd`.
2. Déclare le fournisseur d'identité HTPasswd dans l'OAuth du cluster.
3. Donne le rôle `cluster-admin` à `student`.

VÉRIFICATION

`oc login -u student` puis `oc get nodes` fonctionne ; `oc login -u dev` puis `oc get nodes` est refusé.

SOLUTION

```
$ htpasswd -c -B -b users.htpasswd student <motdepasse1>
$ htpasswd -B -b users.htpasswd dev <motdepasse2>
$ oc create secret generic htpass-secret --from-file=htpasswd=users.htpasswd -n openshift-
config
$ oc edit oauth cluster
spec:
  identityProviders:
    - name: local
      mappingMethod: claim
      type: HTPasswd
      htpasswd:
        fileData:
          name: htpass-secret
$ oc get pods -n openshift-authentication -w      # attendre le redémarrage
$ oc adm policy add-cluster-role-to-user cluster-admin student
```

E03 Tour de la console web

EX180 · 1.2

*Savoir retrouver l'état du cluster dans la console, sans CLI.***ÉNONCÉ**

1. Vue **Administrator** : trouve la version du cluster, l'état des ClusterOperators, les alertes actives et l'utilisation CPU/mémoire du nœud.
2. Vue **Developer** (ou « Topology ») : crée un projet `console-test`.
3. Ouvre le **terminal web** ou la page d'un pod et consulte ses logs.

VÉRIFICATION

Tu sais répondre sans CLI : version exacte ? combien d'alertes en « Firing » ? quel ClusterOperator est le plus récent à avoir changé d'état ?

CHEMINS

- Home > Overview : version, état, utilisation. Administration > Cluster Settings : version et ClusterOperators.
- Observe > Alerting : alertes. Observe > Metrics : requêtes PromQL.
- Workloads > Pods > (pod) > Logs / Terminal / Events.

E04 Évaluer la santé du cluster en CLI

EX180 · 1.2

*Diagnostiquer rapidement l'état de la plateforme.***ÉNONCÉ**

1. Affiche l'état du nœud, sa charge et ses rôles.
2. Liste les ClusterOperators qui ne sont pas `Available=True`.
3. Affiche la version du cluster et son canal de mise à jour.
4. Liste les 10 derniers événements de type `Warning` du cluster.

VÉRIFICATION

Tu obtiens chaque information en une seule commande.

SOLUTION

```
$ oc get nodes -o wide; oc adm top nodes
$ oc get co | grep -v "True         False         False"
$ oc get clusterversion; oc get clusterversion version -o jsonpath='{.spec.channel}{ "\n"}'
$ oc get events -A --field-selector type=Warning --sort-by=.lastTimestamp | tail -10
```

E05 Interroger l'API : jsonpath et custom-columns

EX180 · 1.2

Extraire exactement l'information demandée (fréquent à l'examen : « écris dans /home/student/x.txt la liste de... »).

ÉNONCÉ

1. Liste le nom de tous les pods du namespace `openshift-console`, un par ligne.
2. Affiche un tableau NOM / IMAGE de tous les deployments de `openshift-console`.
3. Écris dans `~/pods-console.txt` le nom et le nœud de chaque pod de ce namespace.

SOLUTION

```
$ oc get pods -n openshift-console -o jsonpath='{range .items[*]}{.metadata.name}{ "\n"}{end}'
$ oc get deploy -n openshift-console \
  -o custom-columns=NOM:.metadata.name,IMAGE:.spec.template.spec.containers[0].image
$ oc get pods -n openshift-console \
  -o custom-columns=NOM:.metadata.name,NODE:.spec.nodeName --no-headers > ~/pods-console.txt
```

E06 Découvrir les ressources de l'API

EX180 · 1.2

Trouver seul le bon objet et le bon champ, comme avec la doc fournie à l'examen.

ÉNONCÉ

1. Liste les ressources du groupe `route.openshift.io`.
2. Trouve le champ qui définit le type de terminaison TLS d'une Route.
3. Trouve le chemin YAML pour définir la mémoire maximale d'un conteneur dans un Deployment.

SOLUTION

```
$ oc api-resources --api-group=route.openshift.io
$ oc explain route.spec.tls.termination
$ oc explain deployment.spec.template.spec.containers.resources.limits
```

E07 Logs et événements d'un composant

EX180 · 1.2

*Lire les logs des composants de la plateforme.***ÉNONCÉ**

1. Affiche les 20 dernières lignes de log du router (namespace `openshift-ingress`).
2. Affiche les logs du service kubelet du nœud.
3. Même chose dans la console web.

SOLUTION

```
$ oc get pods -n openshift-ingress
$ oc logs -n openshift-ingress deploy/router-default --tail=20
$ oc adm node-logs <nœud> -u kubelet | tail -20
```

E08 Collecter les données de support

EX180 · 1.3

*Produire l'archive que demande le support Red Hat lors d'un incident.***ÉNONCÉ**

1. Lance une collecte `must-gather` complète dans `~/mg`.
2. Crée une archive `must-gather.tar.gz` de ce dossier.
3. Fais une collecte ciblée du ClusterOperator `ingress` avec `oc adm inspect`.
4. Récupère l'identifiant du cluster (demandé dans tout ticket).

VÉRIFICATION

L'archive existe et contient un dossier par image de collecte ; `~/inspect` contient les ressources du namespace `openshift-ingress-operator`.

SOLUTION

```
$ oc adm must-gather --dest-dir=$HOME/mg # plusieurs minutes
$ tar czf must-gather.tar.gz -C $HOME mg
$ oc adm inspect clusteroperator/ingress --dest-dir=$HOME/inspect
$ oc get clusterversion -o jsonpath='{.items[0].spec.clusterID}'
```

NOTE

Il existe des images `must-gather` spécifiques à certains Operators (`--image=...`) ; la documentation de chaque Operator indique laquelle utiliser.

■ Bloc B – Conteneurs et pods (DO180 ch. 3)

E09 Pod non géré

EX180 · 2.1

Créer un pod simple et interagir avec lui.

ÉNONCÉ

1. Crée le projet `pods`.
2. Lance un pod `web` avec l'image `registry.access.redhat.com/ubi9/httpd-24`.
3. Affiche ses logs, son IP, puis exécute `cat /etc/redhat-release` dedans.
4. Supprime-le : que se passe-t-il ? Pourquoi ?

SOLUTION

```
$ oc new-project pods
$ oc run web --image=registry.access.redhat.com/ubi9/httpd-24
$ oc get pod web -o wide; oc logs web
$ oc exec web -- cat /etc/redhat-release
$ oc delete pod web          # il disparaît : aucun contrôleur ne le recrée
```

E10 Pod interactif et diagnostic réseau

EX180 · 2.1

Utiliser un pod temporaire comme boîte à outils.

ÉNONCÉ

1. Relance le pod `web` de E09.
2. Démarre un pod temporaire interactif UBI qui se supprime à la sortie.
3. Depuis ce pod, fais un `curl` vers l'IP de `web` sur le port 8080.

SOLUTION

```
$ oc run web --image=registry.access.redhat.com/ubi9/httpd-24
$ IP=$(oc get pod web -o jsonpath='{.status.podIP}')
$ oc run -it --rm tools --image=registry.access.redhat.com/ubi9/ubi -- curl -s http://$IP:8080 | head
```

E11 Pod depuis un manifeste YAML

EX180 · 2.1

*Générer puis modifier un manifeste plutôt que tout écrire à la main.***ÉNONCÉ**

1. Génère le YAML d'un pod `db-client` (image UBI, commande `sleep infinity`) sans le créer.
2. Ajoute-lui le label `role=client` et la variable `MODE=test`.
3. Applique-le et vérifie la variable.

SOLUTION

```
$ oc run db-client --image=registry.access.redhat.com/ubi9/ubi \
  --dry-run=client -o yaml -- sleep infinity > pod.yaml
# éditer : metadata.labels.role: client ; spec.containers[0].env: [{name: MODE, value:
test}]
$ oc apply -f pod.yaml
$ oc exec db-client -- env | grep MODE; oc get pod -l role=client
```

E12 Dépanner un pod qui échoue (SCC)

EX180 · 2.1

*Comprendre le piège n°1 d'OpenShift : les images qui exigent root.***ÉNONCÉ**

1. Lance un pod `nginx` avec l'image Docker Hub `docker.io/library/nginx`.
2. Observe son état et trouve la cause avec les logs.
3. Remplace-le par une image compatible (non root, port 8080) : `quay.io/redhattraining/hello-world-nginx`.

SOLUTION

```
$ oc run nginx --image=docker.io/library/nginx
$ oc get pod nginx          # CrashLoopBackOff
$ oc logs nginx             # Permission denied (UID aléatoire, pas de root, port 80
interdit)
$ oc get pod nginx -o jsonpath='{.metadata.annotations.openshift\.io/scc}{"\n"}' #
restricted-v2
$ oc delete pod nginx
$ oc run nginx --image=quay.io/redhattraining/hello-world-nginx
```

À RETENIR

À l'examen, on ne te demandera pas d'élèver les droits (c'est du niveau 2 : SCC `anyuid` + ServiceAccount). La bonne réponse au niveau 1 est d'utiliser une image adaptée (UBI, images Red Hat, images « non-root »).

E13 Trouver et inspecter une image

EX180 · 2.2

Connaître le contenu d'une image avant de la déployer.

ÉNONCÉ

1. Pour `registry.access.redhat.com/ubi9/httpd-24` : trouve l'utilisateur, les ports exposés, la commande de démarrage et le digest.
2. Liste les ImageStreams fournies par OpenShift et les tags de `httpd`.
3. Cherche une image PostgreSQL sur `catalog.redhat.com`.

SOLUTION

```
$ oc image info registry.access.redhat.com/ubi9/httpd-24 --filter-by-os=linux/amd64
$ skopeo inspect docker://registry.access.redhat.com/ubi9/httpd-24 | jq '.Digest,.Labels'
$ oc get is -n openshift
$ oc get istag -n openshift | grep httpd
$ podman inspect registry.access.redhat.com/ubi9/httpd-24 --format '{{.Config.User}}
{{.Config.ExposedPorts}}' # après podman pull
```

E14 Tags et digests

EX180 · 2.2

Savoir pourquoi un digest est plus sûr qu'un tag.

ÉNONCÉ

1. Récupère le digest actuel du tag `latest` de `ubi9/ubi`.
2. Lance un pod en référençant l'image **par digest**.
3. Explique en une phrase la différence tag / digest.

SOLUTION

```
$ D=$(skopeo inspect docker://registry.access.redhat.com/ubi9/ubi:latest | jq -r .Digest)
$ oc run pinned --image=registry.access.redhat.com/ubi9/ubi@$D -- sleep infinity
$ oc get pod pinned -o jsonpath='{.status.containerStatuses[0].imageID}{"\n"}'
```

Un tag est une étiquette mobile (peut pointer demain vers une autre image) ; un digest (sha256) désigne un contenu unique et immuable.

■ Bloc C – Déployer et exposer (DO180 ch. 4)

E15 Déployer depuis une image avec oc new-app

EX180 · 2.3

La méthode la plus rapide à l'examen.

ÉNONCÉ

1. Crée le projet `app1`.
2. Déploie `quay.io/redhattraining/hello-world-nginx` sous le nom `hello`.
3. Liste tout ce que `new-app` a créé.

SOLUTION

```
$ oc new-project app1
$ oc new-app --image=quay.io/redhattraining/hello-world-nginx --name=hello
$ oc get all -l app=hello      # deployment, replicaset, pod, service (+ imagestream)
```

E16 Déployer depuis un manifeste (Deployment + Service)

EX180 · 2.3

Maîtriser la méthode déclarative, reproductible.

ÉNONCÉ

1. Dans `app1`, génère un Deployment `api` (image `registry.access.redhat.com/ubi9/httpd-24`, 2 réplicas) en YAML.
2. Génère le Service associé (port 8080).
3. Applique les deux fichiers et vérifie les endpoints.

SOLUTION

```
$ oc create deployment api --image=registry.access.redhat.com/ubi9/httpd-24 \
  --replicas=2 --port=8080 --dry-run=client -o yaml > api-deploy.yaml
$ oc apply -f api-deploy.yaml
$ oc expose deploy/api --port=8080 --dry-run=client -o yaml > api-svc.yaml
$ oc apply -f api-svc.yaml
$ oc get endpoints api      # 2 adresses IP:8080
```

E17 Déployer depuis un template

EX180 · 2.3

*Utiliser les templates fournis par OpenShift.***ÉNONCÉ**

1. Liste les templates du namespace `openshift` et trouve une base MySQL **éphémère**.
2. Affiche ses paramètres.
3. Déploie-la dans le projet `tpl` avec l'utilisateur `app`, le mot de passe `redhat` et la base `items`.
4. Connecte-toi à la base depuis le pod.

SOLUTION

```
$ oc get templates -n openshift | grep -i mysql
$ oc process --parameters mysql-ephemeral -n openshift
$ oc new-project tpl
$ oc new-app --template=mysql-ephemeral -p MYSQL_USER=app -p MYSQL_PASSWORD=redhat -p
MYSQL_DATABASE=items
# ou : oc process openshift/mysql-ephemeral -p MYSQL_USER=app ... | oc apply -f -
$ oc rsh deploy/mysql mysql -uapp -predhat items -e 'SHOW TABLES;'
```

NOTE

Les templates disponibles dépendent de l'installation (Samples Operator). Si `mysql-ephemeral` n'existe pas, prends un autre template de la liste : la méthode est identique.

E18 Exposer en HTTP avec une Route

EX180 · 2.4

*Rendre une application accessible depuis ton navigateur.***ÉNONCÉ**

1. Expose le service `hello` (projet `app1`).
2. Affiche l'URL générée et teste-la avec `curl`.
3. Crée une seconde route avec le nom d'hôte `hello.apps.ocp.lab.example.com`.

SOLUTION

```
$ oc expose svc/hello
$ oc get route hello -o jsonpath='{.spec.hosts[0].host}' # hello-
app1.apps.ocp.lab.example.com
$ curl http://hello-app1.apps.ocp.lab.example.com
$ oc expose svc/hello --name=hello-custom --hostname=hello.apps.ocp.lab.example.com
```

E19 Route HTTPS (edge)

EX180 · 2.4

*Exposer en HTTPS avec le certificat du router.***ÉNONCÉ**

1. Crée une route **edge** `hello-secure` vers le service `hello`.
2. Force la redirection HTTP → HTTPS.
3. Teste avec `curl -k`.

SOLUTION

```
$ oc create route edge hello-secure --service=hello --insecure-policy=Redirect
$ curl -k https://$(oc get route hello-secure -o jsonpath='{.spec.host}')
$ curl -I http://$(oc get route hello-secure -o jsonpath='{.spec.host}') # 302 vers
https
```

Types de terminaison : **edge** (TLS au router), **passthrough** (TLS jusqu'au pod), **reencrypt** (TLS au router puis nouveau TLS vers le pod).

E20 Mettre à l'échelle manuellement

EX180 · 2.4

*Changer le nombre de réplicas et observer la répartition du trafic.***ÉNONCÉ**

1. Passe `hello` à 3 réplicas.
2. Vérifie que le Service a 3 endpoints.
3. Supprime un pod : que se passe-t-il ?
4. Reviens à 1 réplica.

SOLUTION

```
$ oc scale deploy/hello --replicas=3
$ oc get pods -l deployment=hello; oc get endpoints hello
$ oc delete pod <un-pod>; oc get pods -w # le ReplicaSet en recrée un
$ oc scale deploy/hello --replicas=1
```

E21 Dépanner une application injoignable

EX180 · 2.4

*Remonter la chaîne Route → Service → Endpoints → Pods.***ÉNONCÉ (SABOTAGE VOLONTAIRE)**

1. Modifie le sélecteur du service `hello` pour qu'il cherche `app=bad`.
2. Constate que la route renvoie une erreur 503.
3. Trouve la panne uniquement avec `oc get / describe` et répare-la.

SOLUTION

```
$ oc patch svc hello -p '{"spec":{"selector":{"app":"bad","deployment":null}}}'
$ curl -I http://hello-app1.apps.ocp.lab.example.com # 503
$ oc get endpoints hello # <none> → le sélecteur ne matche aucun pod
$ oc get pods --show-labels; oc describe svc hello
$ oc patch svc hello -p '{"spec":{"selector":{"app":null,"deployment":"hello"}}}'
# (adapte au label réel des pods, visible avec --show-labels)
```

■ Bloc D – Configuration et stockage (DO180 ch. 5)**E22 ConfigMap en variables et en fichier**

EX180 · 3.1

*Sortir la configuration de l'image.***ÉNONCÉ**

1. Projet `cfg` : déploie `registry.access.redhat.com/ubi9/httpd-24` sous le nom `site` et expose-le.
2. Crée une ConfigMap `site-env` avec `APP_COLOR=red` et `APP_ENV=lab` ; injecte-la en variables.
3. Crée une ConfigMap `site-html` à partir d'un fichier `index.html` et monte-la dans `/var/www/html`.

VÉRIFICATION

```
oc exec deploy/site -- env | grep APP_ ; curl sur la route affiche ton index.html.
```

SOLUTION

```
$ oc new-project cfg
$ oc new-app --image=registry.access.redhat.com/ubi9/httpd-24 --name=site && oc expose svc/site
$ oc create configmap site-env --from-literal=APP_COLOR=red --from-literal=APP_ENV=lab
$ oc set env deploy/site --from=configmap/site-env
$ echo '<h1>Lab OpenShift</h1>' > index.html
$ oc create configmap site-html --from-file=index.html
$ oc set volumes deploy/site --add --name=html --type=configmap \
  --configmap-name=site-html --mount-path=/var/www/html
$ curl http://site-cfg.apps.ocp.lab.example.com
```

E23 Secret pour des identifiants

EX180 · 3.1

Gérer des informations sensibles.

ÉNONCÉ

1. Crée un Secret `db-creds` avec `user=app` et `password=S3cret!`.
2. Injecte-le dans `site` avec le préfixe `DB_`.
3. Décode la valeur du mot de passe depuis le Secret.
4. Change le mot de passe : le pod voit-il la nouvelle valeur sans redémarrer ?

SOLUTION

```
$ oc create secret generic db-creds --from-literal=user=app --from-literal=password='S3cret!'
$ oc set env deploy/site --from=secret/db-creds --prefix=DB_
$ oc extract secret/db-creds --keys=password --to=-
$ oc set data secret/db-creds --from-literal=password=N3w
# Les variables d'env ne changent pas sans redémarrage : oc rollout restart deploy/site
# (un Secret monté en volume, lui, est mis à jour automatiquement après quelques secondes)
```

E24 Volume persistant avec LVMS

EX180 · 3.2

Conserver des données après la suppression d'un pod.

ÉNONCÉ

1. Identifie la StorageClass disponible et si elle est par défaut.
2. Ajoute à `site` un PVC `site-data` de 1 Gi (RWO) monté dans `/data`.
3. Écris un fichier dans `/data`, supprime le pod, vérifie que le fichier est toujours là.

SOLUTION

```
$ oc get sc                                     # ex. lvms-vg1 (default)
$ oc set volumes deploy/site --add --name=data --type=pvc --claim-name=site-data \
    --claim-size=1Gi --claim-mode=rwo --claim-class=lvms-vg1 --mount-path=/data
$ oc get pvc site-data                          # Bound
$ oc exec deploy/site -- sh -c 'date > /data/test.txt'
$ oc delete pod -l deployment=site; oc get pods -w
$ oc exec deploy/site -- cat /data/test.txt
```

PIÈGE

Un volume RWO ne peut être monté que par un seul nœud à la fois. Avec la stratégie `RollingUpdate` et plusieurs nœuds, le nouveau pod peut rester bloqué. Sur ton SNO ça passe, mais en production on utilise `Recreate` ou un `StatefulSet` pour ce cas.

E25 PVC déclaratif et base de données persistante

EX180 · 3.2

*Écrire un PVC en YAML et l'utiliser pour une vraie base.***ÉNONCÉ**

1. Projet `db` : écris et applique un PVC `pg-data` de 2 Gi.
2. Déploie `registry.redhat.io/rhel9/postgresql-16` avec les variables `POSTGRESQL_USER`, `POSTGRESQL_PASSWORD`, `POSTGRESQL_DATABASE` (via un Secret).
3. Monte le PVC dans `/var/lib/pgsql/data`, crée une table, redémarre le pod et vérifie que la table existe.

SOLUTION

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pg-data
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 2Gi
$ oc new-project db && oc apply -f pvc.yaml
$ oc create secret generic pg --from-literal=POSTGRESQL_USER=app \
  --from-literal=POSTGRESQL_PASSWORD=redhat --from-literal=POSTGRESQL_DATABASE=items
$ oc new-app --image=registry.redhat.io/rhel9/postgresql-16 --name=pg
$ oc set env deploy/pg --from=secret/pg
$ oc set volumes deploy/pg --add --name=data --type=pvc --claim-name=pg-data --mount-
path=/var/lib/pgsql/data
$ oc rsh deploy/pg psql -d items -c 'CREATE TABLE t(id int);'
$ oc rollout restart deploy/pg && oc rollout status deploy/pg
$ oc rsh deploy/pg psql -d items -c '\dt'

```

■ Bloc E – Fiabilité (DO180 ch. 6)

E26 Sondes de santé

EX180 · 4.1

Éviter d'envoyer du trafic à un pod pas prêt et redémarrer un pod bloqué.

ÉNONCÉ

1. Sur `hello` (projet `app1`) : readiness probe HTTP GET `/` port 8080, délai initial 5 s.
2. Liveness probe TCP sur 8080 toutes les 10 s.
3. Casse volontairement la readiness (chemin `/nope`) : observe `READY 0/1` et les endpoints, puis répare.

SOLUTION

```
$ oc set probe deploy/hello --readiness --get-url=http://:8080/ --initial-delay-seconds=5
$ oc set probe deploy/hello --liveness --open-tcp=8080 --period-seconds=10
$ oc set probe deploy/hello --readiness --get-url=http://:8080/nope
$ oc get pods; oc get endpoints hello; oc describe pod -l deployment=hello | grep -A3
Readiness
$ oc set probe deploy/hello --readiness --get-url=http://:8080/
# supprimer une probe : oc set probe deploy/hello --liveness --remove
```

E27 Requests et limits

EX180 · 4.2

Réserver et plafonner les ressources.

ÉNONCÉ

1. Sur `hello` : requests 50m CPU / 64Mi, limits 200m CPU / 128Mi.
2. Mets volontairement une request CPU à 64 cœurs : quel état ? pourquoi ?
3. Remets des valeurs raisonnables et affiche la consommation réelle des pods.

SOLUTION

```
$ oc set resources deploy/hello --requests=cpu=50m,memory=64Mi --
limits=cpu=200m,memory=128Mi
$ oc set resources deploy/hello --requests=cpu=64 --limits=cpu=64
$ oc get pods; oc describe pod <nouveau-pod> | tail # Pending : Insufficient cpu
$ oc set resources deploy/hello --requests=cpu=50m,memory=64Mi --
limits=cpu=200m,memory=128Mi
$ oc adm top pods
```

E28 Autoscaling horizontal (HPA)

EX180 · 4.3

*Adapter automatiquement le nombre de pods à la charge.***ÉNONCÉ**

1. Crée un HPA pour `hello` : min 1, max 4, cible 50 % CPU.
2. Génère de la charge et observe la montée en réplicas.
3. Arrête la charge et observe la redescende (quelques minutes).

SOLUTION

```
$ oc autoscale deploy/hello --min=1 --max=4 --cpu-percent=50
$ oc get hpa hello -w &
$ oc run -it --rm load --image=registry.access.redhat.com/ubi9/ubi -- \
    bash -c 'while true; do curl -s http://hello:8080 > /dev/null; done'
# Ctrl+C pour arrêter la charge
```

IMPORTANT

Sans **requests CPU** sur le conteneur, l'HPA affiche `<unknown>` et ne fait rien. C'est la première chose à vérifier.

■ Bloc F – Mises à jour (DO180 ch. 7)**E29 Rollout et rollback**

DO180 · ch. 7

*Mettre à jour une application et revenir en arrière proprement.***ÉNONCÉ**

1. Projet `upd` : déploie `registry.access.redhat.com/ubi9/httpd-24:1-300` (ou un tag ancien existant) sous le nom `web`.
2. Mets à jour vers le tag `latest` et suis le déploiement.
3. Mets à jour vers un tag inexistant : observe le blocage.
4. Reviens à la révision qui fonctionnait.

SOLUTION

```
$ skopeo list-tags docker://registry.access.redhat.com/ubi9/httpd-24 | jq -r '.Tags[]' | tail
$ oc new-project upd
$ oc create deployment web --image=registry.access.redhat.com/ubi9/httpd-24:<tag-ancien>
$ oc set image deploy/web httpd-24=registry.access.redhat.com/ubi9/httpd-24:latest
$ oc rollout status deploy/web; oc rollout history deploy/web
$ oc set image deploy/web httpd-24=registry.access.redhat.com/ubi9/httpd-24:nexistepas
$ oc get pods # ImagePullBackOff ; l'ancien pod continue de servir
$ oc rollout undo deploy/web
# Nom du conteneur : oc get deploy web -o jsonpath='{.spec.template.spec.containers[*].name}'
```

E30 ImageStream et déclencheur automatique

DO180 · ch. 7

Redéployer automatiquement quand une image change.

ÉNONCÉ

1. Crée une ImageStream `web-is` dont le tag `prod` pointe vers une version de httpd-24.
2. Configure le Deployment `web` pour suivre `web-is:prod`.
3. Fais pointer `prod` vers une autre version : le Deployment doit se mettre à jour seul.

SOLUTION

```
$ oc import-image web-is:prod --from=registry.access.redhat.com/ubi9/httpd-24:<tag-A> --confirm
$ oc set triggers deploy/web --from-image=web-is:prod -c httpd-24
$ oc tag registry.access.redhat.com/ubi9/httpd-24:<tag-B> web-is:prod
$ oc rollout history deploy/web # nouvelle révision créée automatiquement
$ oc get istag web-is:prod
```

POUR ALLER PLUS LOIN

Refais E15 à E28 avec une de tes propres applications (par exemple une API + PostgreSQL) : Deployment, Service, Route edge, ConfigMap, Secret, PVC, probes sur un endpoint `/healthz`, requests/limits et HPA. C'est le meilleur projet à montrer en entretien.

CHAPITRE 08

Examen blanc (2 heures)

Conditions réelles : chronomètre, pas de notes, uniquement docs.redhat.com et `oc explain`.

CONSIGNES

- Supprime d'abord tous les projets d'exercices (`oc delete project app1 cfg db upd tpl pods`).
- Tout doit **survivre à un redémarrage des pods** (règle de persistance). Vérifie à la fin avec `oc delete pod --all -n <projet>`.
- Barème indicatif : chaque tâche vaut 1 point ; objectif $\geq 11/14$ ($\approx 80\%$).

#	Tâche	Objectif
1	Écris dans <code>~/exam/version.txt</code> la version exacte du cluster, et dans <code>~/exam/co.txt</code> le nom de chaque ClusterOperator, un par ligne.	1.2
2	Produis une collecte must-gather dans <code>~/exam/mg</code> .	1.3
3	Crée le projet <code>exam-shop</code> .	1.2
4	Dans ce projet, lance un pod non géré <code>debug</code> (image UBI 9) qui reste actif (<code>sleep infinity</code>) et porte le label <code>tier=tools</code> .	2.1
5	Écris dans <code>~/exam/image.txt</code> l'utilisateur et le digest de l'image <code>registry.access.redhat.com/ubi9/httpd-24:latest</code> .	2.2
6	Déploie l'application <code>front</code> à partir de l'image <code>quay.io/redhattraining/hello-world-nginx</code> , avec 2 réplicas.	2.3
7	Déploie une base à partir du template MySQL (ou PostgreSQL) éphémère, avec utilisateur <code>shop</code> , mot de passe <code>shop123</code> , base <code>catalog</code> .	2.3
8	Rends <code>front</code> accessible en HTTPS sur <code>https://shop.apps.ocp.lab.example.com</code> (terminaison edge, redirection HTTP → HTTPS).	2.4
9	Crée une ConfigMap <code>front-cfg</code> (<code>THEME=dark</code> , <code>LANG=fr</code>) et injecte-la en variables dans <code>front</code> .	3.1
10	Stocke les identifiants de la base dans un Secret <code>shop-db</code> et injecte-le dans <code>front</code> avec le préfixe <code>DB_</code> .	3.1
11	Ajoute à <code>front</code> un volume persistant de 1 Gi, nommé <code>front-uploads</code> , monté dans <code>/</code> <code>uploads</code> .	3.2
12	Configure sur <code>front</code> une readiness probe HTTP sur <code>/</code> port 8080 et une liveness probe TCP sur 8080.	4.1
13	Définit pour <code>front</code> : requests 50m CPU / 64Mi, limits 250m CPU / 256Mi.	4.2

#	Tâche	Objectif
14	Configure l'autoscaling de <code>front</code> entre 2 et 5 réplicas à 60 % de CPU.	4.3

■ Grille d'auto-correction

```
$ cat ~/exam/version.txt ~/exam/co.txt; ls ~/exam/mg
$ oc get pod debug -n exam-shop --show-labels
$ oc get deploy,svc,route,cm,secret,pvc,hpa -n exam-shop
$ curl -kI https://shop.apps.ocp.lab.example.com; curl -I http://
shop.apps.ocp.lab.example.com # 200 puis 302
$ oc set env deploy/front --list -n exam-shop | grep -E 'THEME|LANG|DB_'
$ oc get deploy front -n exam-shop -o jsonpath='{.spec.template.spec.containers[0].resources}
{"\n"}'
$ oc describe deploy front -n exam-shop | grep -E 'Liveness|Readiness|/uploads'
$ oc delete pod --all -n exam-shop && oc get pods -n exam-shop -w # tout revient Ready ?
```

Attention : le pod non géré `debug` ne revient pas après la suppression de tous les pods. C'est normal et c'est une bonne question de révision : pourquoi ?

CHAPITRE 09

Stratégie d'examen et pièges classiques

Ce qui fait perdre des points à des candidats qui connaissent pourtant la matière.

■ Avant l'examen

- ☐ Relire la page officielle EX180 et le document des objectifs par version (LMS).
- ☐ Relire le Red Hat Certification Program Guide (conditions, pièces d'identité, examen à distance).
- ☐ Savoir naviguer dans **docs.redhat.com** hors connexion à Internet : quels chapitres contiennent les exemples YAML de Route, PVC, probes, HPA.
- ☐ Refaire l'examen blanc la veille, en moins de 1 h 30.

■ Pendant l'examen

1. **Lis toutes les tâches d'abord** (5 min). Repère les dépendances (ex. le Secret doit exister avant d'être injecté) et les tâches rapides.
2. **Vérifie le projet courant** avant chaque commande : `oc project`. Créer une ressource dans le mauvais namespace = tâche à zéro.
3. **Respecte les noms à la lettre** : nom de ressource, de clé, de label, de chemin. La correction est automatique.
4. **Préfère les commandes impératives** (`oc new-app`, `oc set ...`, `oc create ... --dry-run=client -o yaml`) : plus rapides et moins d'erreurs d'indentation.
5. **Vérifie chaque tâche immédiatement** (`get`, `describe`, `curl`) avant de passer à la suivante.
6. **Ne reste pas bloqué** : au-delà de 10 minutes, passe et reviens plus tard.
7. **Test de persistance final** : supprime les pods de tes projets et vérifie que tout revient.

■ Pièges classiques

Piège	Comment l'éviter
Image qui exige root (nginx, httpd de Docker Hub)	Utiliser les images UBI / Red Hat ou non-root ; lire <code>oc logs</code> .
Port < 1024 dans le conteneur	Les images compatibles écoutent sur 8080/8443 ; le Service et la Route doivent cibler ce port.
HPA en <code><unknown></code>	Définir des requests CPU sur le conteneur.
Variables d'env non mises à jour	Une modification de ConfigMap/Secret injectée en env demande un <code>oc rollout restart</code> .
Route en 503	Service sans endpoints : sélecteur faux, pods non Ready (readiness) ou mauvais <code>targetPort</code> .

Piège	Comment l'éviter
Route en « Application is not available »	Même diagnostic ; vérifier aussi le nom d'hôte et le DNS.
PVC en <code>Pending</code>	Mauvaise StorageClass, taille ou mode d'accès non supporté (RWX sur LVMS).
Pod non géré au lieu d'un Deployment	Si la tâche demande une « application », créer un Deployment (sinon elle ne survit pas).
Oubli du namespace	<code>-n <projet></code> systématique ou <code>oc project</code> en début de tâche.
Fichier de réponse mal placé	Écrire exactement au chemin demandé, avec le format demandé (un élément par ligne...).
Nom de conteneur inconnu pour <code>oc set image</code>	<code>oc get deploy x -o jsonpath='{.spec.template.spec.containers[*].name}'</code> .

CHAPITRE 10

Plan de révision, checklist et ressources

Un plan sur 4 semaines, compatible avec la durée d'essai de ton cluster.

■ Plan sur 4 semaines

Semaine	Thèmes	À faire
1	Plateforme et CLI · pods · images	Chapitres 1 à 4 de ce guide ; exercices E01 à E14 ; console web en parallèle.
2	Déployer et exposer · configuration	E15 à E23 ; refaire E15 à E21 de mémoire en fin de semaine.
3	Stockage · fiabilité · mises à jour	E24 à E30 ; déployer un projet personnel sur le cluster.
4	Révision et examens blancs	Examen blanc ×2 (objectif < 1 h 30) ; relire le chapitre 9 ; réserver l'examen.

■ Checklist de compétences

GÉRER OPENSIFT

- ☐ Installer oc et se connecter (token, utilisateur)
- ☐ Lire l'état du cluster en CLI et dans la console
- ☐ Filtrer avec jsonpath et custom-columns
- ☐ Utiliser `oc explain` et `api-resources`
- ☐ Lire logs et événements
- ☐ Produire un must-gather et un inspect

DÉPLOYER

- ☐ Créer un pod, un pod interactif, un pod YAML
- ☐ Diagnostiquer CrashLoopBackOff / ImagePullBackOff
- ☐ Inspecter une image (user, ports, digest)
- ☐ Déployer depuis une image (`new-app`, `create deployment`)
- ☐ Déployer depuis un template
- ☐ Créer Service et Route (HTTP, edge)
- ☐ Mettre à l'échelle manuellement

CONFIGURATION ET STOCKAGE

- ☐ ConfigMap en variables et en fichiers
- ☐ Secret en variables (avec préfixe) et décodage
- ☐ PVC impératif (`oc set volumes`) et YAML
- ☐ Vérifier la persistance des données

FIABILITÉ ET MISES À JOUR

- ☐ Readiness, liveness (HTTP, TCP)
- ☐ Requests et limits ; diagnostiquer Pending / OOMKilled
- ☐ HPA fonctionnel
- ☐ Mise à jour d'image, rollout status/history/undo
- ☐ ImageStream et trigger automatique

EXAMEN

- ☐ Examen blanc réussi en < 2 h
- ☐ Examen blanc réussi en < 1 h 30

■ Ressources

Ressource	Adresse / usage
Page officielle de l'examen EX180	<code>redhat.com/en/services/training/ex180-red-hat-certified-technologist-in-openshift-exam</code>
Page officielle de l'examen EX280 (niveau 2)	<code>redhat.com/en/services/training/red-hat-certified-openshift-administrator-exam</code>
Documentation OpenShift (celle de l'examen)	<code>docs.redhat.com</code> > OpenShift Container Platform > 4.22
Programme de certification	Red Hat Certification Program Guide (<code>docs.redhat.com</code>)
Évaluation de niveau gratuite	<code>skills.ole.redhat.com</code>
Labs interactifs gratuits	<code>developers.redhat.com/learn/openshift</code> · <code>redhat.com/en/interactive-labs</code>
Developer Sandbox (cluster gratuit en ligne)	<code>developers.redhat.com/developer-sandbox</code>
Catalogue d'images Red Hat	<code>catalog.redhat.com</code>
Client <code>oc</code>	<code>mirror.openshift.com/pub/openshift-v4/clients/ocp/stable/</code>
Version communautaire gratuite	OKD (<code>okd.io</code>) – à installer en SNO après l'essai de 60 jours

SOURCES DE CE GUIDE

Pages publiques Red Hat de l'EX180 et de l'EX280 (consultées le 30/09/2026), plan public du cours DO180 (4.18) diffusé par les partenaires de formation, documentation OpenShift et configuration réelle de ton lab. Les exercices et l'examen blanc sont originaux : ils ne reproduisent pas les labs officiels Red Hat.